Estimation Module - Developer Knowledge Base

1. Executive Summary

The Estimation module provides a comprehensive sales estimation workflow for jewelry retail operations. It enables sales staff to create price quotations for customers by combining tagged inventory items, catalog products, custom items, and order-based items while processing old metal exchanges (purchases), applying discounts, and calculating taxes.

Primary User Roles: - Sales Staff: Create and manage estimations - Branch Managers: Review estimations, manage EDA approvals - Accounts/Admin: Process estimations to bills

Top-Level Outcomes: - Generate accurate price estimations for customer purchases - Handle old metal (gold/silver) exchange transactions - Enforce discount approval workflows via EDA (Estimate Discount Approval) - Prepare estimations for conversion to billing

---

2. Module Metadata & Mapping Table

2.1 Logical Component Descriptions

Component	Description
EstimationEngine	Core controller handling estimation CRUD operations, item processing, and PDF generation
EDAEngine	Discount approval workflow controller for estimations requiring manager approval
EstimationRepository	Data access layer for estimation persistence and retrieval
EDARepository	Data access layer for EDA approval queue and updates
EstimationUI	Client-side logic for form interactions, calculations, and AJAX operations
EDAUI	Client-side logic for approval/rejection workflows

2.2 Codebase Mapping Table

Logical Component	Codebase Location	Type
EstimationEngine	`admin/application/controllers/admin_ret_estimation.php`	Controller
EDAEngine	`admin/application/controllers/admin_ret_eda.php`	Controller
EstimationRepository	`admin/application/models/ret_estimation_model.php`	Model
EDARepository	`admin/application/models/ret_eda_model.php`	Model
AddEstimationView	`admin/application/views/estimation/form.php`	View
EstimationListView	`admin/application/views/estimation/list.php`	View
EDAListView	`admin/application/views/estimation/eda/list.php`	View
EstimationUI	`admin/assets/js/ret_estimation.js`	JavaScript
EDAUI	`admin/assets/js/ret_eda.js`	JavaScript

2.3 Database Tables

Table	Purpose
`ret_estimation`	Master estimation records
`ret_estimation_items`	Estimation line items (tags, catalog, custom, orders)
`ret_estimation_item_stones`	Stone details per estimation item
`ret_estimation_other_charges`	Additional charges per estimation item
`ret_estimation_old_metal_sale_details`	Old metal purchase records within estimation
`ret_esti_old_metal_stone_details`	Stones in old metal items
`ret_est_gift_voucher_details`	Gift voucher utilization
`ret_est_chit_utilization`	Scheme account utilization
`ret_est_sales_return_utilization`	Sales return credit utilization
`ret_est_tag_merge`	Tag merge relationships
`ret_est_other_metals`	Other metal components in tagged items

Table	Purpose
`ret_estimation_item_other_materials`	Other material details

---

3. Pseudocode Conventions

3.1 Function Signature Format

```
FUNCTION FunctionName(param1: Type, param2: Type) -> ReturnType
```

3.2 ID Tag Format

Every pseudocode function includes a unique identifier:

```
#ID: ESTIMATION.<SUBMODULE>.<FUNCTION_NAME>
```

3.3 Metadata Comment Format

Every function includes a JSON metadata line:

```
#META: {"inputs":["param1","param2"],"outputs":["returnValue"],"critical":true|false}
```

3.4 Precondition/Postcondition Format

```
PRECONDITION: <condition that must be true before execution>
POSTCONDITION: <condition guaranteed after successful execution>
SIDE_EFFECTS: <external state changes>
FAILURE_MODES: <possible failure scenarios>
```

3.5 Control Flow Notation

- `IF...THEN...ELSE...END IF` - Conditional branching
- `FOR EACH item IN collection DO...END FOR` - Iteration
- `WHILE condition DO...END WHILE` - Loop
- `TRY...CATCH...END TRY` - Error handling
- `TRANSACTION BEGIN...COMMIT/ROLLBACK` - Database transactions

3.6 Async Operations

```
ASYNC CALL endpoint WITH parameters -> callback
```

3.7 Event Handling

```
ON EVENT "event_name" DO
    <handler logic>
END ON
```

---

4. Module Overview (Developer View)

4.1 Business Purpose

The Estimation module serves as the pre-billing quotation system in the retail jewelry workflow. It calculates final prices based on: - Metal weight and purity rates - Wastage/making charges - Stone and other material values - Applicable taxes - Old metal exchange credits - Scheme/chit utilization - Discounts (with approval workflow)

4.2 Functional Users

User Type	Capabilities
Sales Staff	Create estimations, add items, apply within-limit discounts
Senior Sales	Apply higher discounts (per employee settings)
Branch Manager	Approve/reject EDA requests, view all branch estimations
Admin	Full access, configure settings

4.3 Lifecycle Stages

```
[Customer Inquiry] -> [Add Estimation] -> [Review/Edit] -> [EDA Approval (if needed)] -> [Convert to
Bill]
                                          ^                    |
                                          |____ Rejection _____|
```

4.4 EDA Approval States

State	`is_eda_approved` Value	Description
Pending	0	Awaiting manager approval
Approved	1	Discount approved, tags marked as sold (status=10)
Rejected	2	Discount rejected, estimation remains editable

4.5 Functional Dependencies

- **Upstream:** Customer Master, Product Master, Tag Master (ret_taging), Rate Master, Settings
- **Downstream:** Billing Module, Reports, Day Closing

---

5. End-to-End Functional Flow

5.1 High-Level Process Narrative

1. **Initiation**: Sales staff opens Add Estimation form, system loads current rates and employee settings
2. **Customer Selection**: Existing customer selected or new customer created inline
3. **Item Addition**: One or more items added via:
    - Tag scan/search (tagged inventory)
    - Catalog selection (non-tagged standard products)
    - Custom item entry (ad-hoc items)
    - Order linkage (customer orders)
4. **Old Metal Processing**: If customer exchanges old gold/silver, details captured
5. **Discount Application**: Line-level or bulk discounts applied within limits
6. **EDA Routing**: If discounts exceed employee limits, estimation flagged for EDA
7. **Save**: Estimation persisted with all related records
8. **Approval** (EDA only): Manager reviews and approves/rejects
9. **Billing Conversion**: Approved estimation converted to bill

5.2 Top-Level Process Pseudocode

```
#ID: ESTIMATION.CORE.MAIN_PROCESS
#META: {"inputs":["user_session","customer_data","items","old_metal","discounts"],"outputs":
["estimation_id"],"critical":true}

FUNCTION ProcessEstimation(user_session, customer_data, items, old_metal, discounts) -> estimation_id

PRECONDITION: User has estimation create permission, branch day is open
```

```
POSTCONDITION: Estimation saved with all child records
SIDE_EFFECTS: Tag statuses may change (if non-EDA), customer may be created
FAILURE_MODES: Day closed, invalid rates, permission denied, transaction failure

BEGIN
    // Phase 1: Initialization
    branch_data := GET_BRANCH_DAY_STATUS(user_session.branch_id)
    IF branch_data.is_day_closed = TRUE THEN
        RAISE ERROR "Day is closed for this branch"
    END IF

    current_rates := LOAD_CURRENT_RATES()
    employee_settings := LOAD_EMPLOYEE_SETTINGS(user_session.employee_id)

    // Phase 2: Customer Resolution
    IF customer_data.is_new = TRUE THEN
        customer_id := CREATE_NEW_CUSTOMER(customer_data)
    ELSE
        customer_id := customer_data.existing_id
    END IF

    // Phase 3: EDA Determination
    requires_eda := CALCULATE_EDA_REQUIREMENT(items, discounts, employee_settings)

    // Phase 4: Estimation Creation
    TRANSACTION BEGIN
        estimation := BUILD_ESTIMATION_RECORD(
            customer_id,
            user_session,
            branch_data,
            current_rates,
            requires_eda
        )
        estimation_id := INSERT_ESTIMATION(estimation)

        // Phase 5: Items Processing
        FOR EACH item IN items DO
            item_id := SAVE_ESTIMATION_ITEM(estimation_id, item)
            SAVE_ITEM_STONES(item_id, item.stones)
            SAVE_ITEM_CHARGES(item_id, item.charges)
            SAVE_ITEM_OTHER_METALS(item_id, item.other_metals)

            IF requires_eda = FALSE AND item.tag_id IS NOT NULL THEN
                UPDATE_TAG_STATUS(item.tag_id, status=9) // Estimated status
            END IF
        END FOR

        // Phase 6: Old Metal Processing
        FOR EACH old_metal_item IN old_metal DO
            SAVE_OLD_METAL_RECORD(estimation_id, old_metal_item)
            SAVE_OLD_METAL_STONES(old_metal_item)
        END FOR
```

```
        // Phase 7: Ancillary Records
        SAVE_GIFT_VOUCHERS(estimation_id, vouchers)
        SAVE_CHIT_UTILIZATION(estimation_id, chit_details)
        SAVE_SALES_RETURN_UTILIZATION(estimation_id, sr_details)

    TRANSACTION COMMIT

    LOG_ACTIVITY(estimation_id, "Estimation Created")
    RETURN estimation_id

EXCEPTION
    TRANSACTION ROLLBACK
    LOG_ERROR(exception)
    RAISE exception
END FUNCTION
```

---

6. Sub-Module: Add Estimation

Route: `admin_ret_estimation/estimation/add`

6.1 Purpose

Provides the primary interface for creating new sales estimations. Handles multi-item entry, pricing calculations, discount application, and old metal exchange processing.

6.2 Functional Actors

Actor	Actions
Sales Staff	Enter items, apply discounts, select customer, save estimation
System	Calculate rates, validate limits, generate estimation number

6.3 Functional Workflow

6.3.1 Form Initialization

```
#ID: ESTIMATION.ADD.INITIALIZE_FORM
#META: {"inputs":["session"],"outputs":["form_data"],"critical":false}

FUNCTION InitializeEstimationForm(session) -> form_data

PRECONDITION: User authenticated with estimation/add permission
POSTCONDITION: Form loaded with all required settings and empty record
SIDE_EFFECTS: None
FAILURE_MODES: Permission denied, settings not found
```

```
BEGIN
    // Load profile and employee settings
    profile := GET_PROFILE_SETTINGS(session.profile_id)
    emp_settings := GET_EMPLOYEE_SETTINGS(session.employee_id)

    // Load rate limits and settings
    settings := {
        min_old_gold_rate: GET_SETTING("min_old_gold_rate"),
        max_old_gold_rate: GET_SETTING("max_old_gold_rate"),
        min_old_silver_rate: GET_SETTING("min_old_silver_rate"),
        max_old_silver_rate: GET_SETTING("max_old_silver_rate"),
        max_cash_allowed: GET_SETTING("max_cash_allowed"),
        weight_scheme_cal_type: GET_SETTING("weightschemecaltype"),
        est_emp_select_req: GET_SETTING("est_emp_select_req"),
        bulk_wastage_discount: GET_SETTING("bulk_wastage_discount"),
        wastage_rate_type: GET_SETTING("wastage_rate_type"),
        tag_split_weight_type: GET_SETTING("tag_split_weight_type")
    }

    // Load reference data
    uom_list := GET_UOM_DETAILS()
    stone_disc_per := GET_STONE_DISCOUNT_PERCENTAGE()

    // Initialize empty estimation record
    estimation := CREATE_EMPTY_ESTIMATION_RECORD()
    estimation.datetime := CURRENT_TIMESTAMP()
    estimation.financial_year := GET_ACTIVE_FINANCIAL_YEAR()

    // Load access permissions
    access := GET_ACCESS_PERMISSIONS("admin_ret_estimation/estimation/add")

    RETURN {
        profile: profile,
        emp_settings: emp_settings,
        settings: settings,
        estimation: estimation,
        uom: uom_list,
        access: access
    }
END FUNCTION
```

6.3.2 Tag Search and Addition

```
#ID: ESTIMATION.ADD.SEARCH_TAG
#META: {"inputs":["search_text","branch_id"],"outputs":["tag_data"],"critical":false}

FUNCTION SearchTagForEstimation(search_text, branch_id) -> tag_data

PRECONDITION: Branch selected, search text length >= 3
POSTCONDITION: Tag details returned if found and available
SIDE_EFFECTS: None
```

```
FAILURE_MODES: Tag not found, tag in wrong branch, tag already sold/estimated

BEGIN
    // Search by tag code or barcode
    tag := QUERY tags WHERE
        (tag_code LIKE search_text OR barcode = search_text)
        AND branch_id = branch_id
        AND tag_status IN (1, 3, 4, 6) // Available statuses

    IF tag IS NULL THEN
        RETURN {status: "NOT_FOUND", message: "Tag not found in this branch"}
    END IF

    IF tag.tag_status = 9 THEN
        RETURN {status: "ALREADY_ESTIMATED", message: "Tag already in estimation"}
    END IF

    IF tag.tag_status = 10 THEN
        RETURN {status: "SOLD", message: "Tag already sold"}
    END IF

    // Load related data
    product := GET_PRODUCT(tag.product_id)
    design := GET_DESIGN(tag.design_id)
    purity := GET_PURITY(tag.purity_id)
    stones := GET_TAG_STONES(tag.tag_id)
    charges := GET_TAG_CHARGES(tag.tag_id)
    other_metals := GET_TAG_OTHER_METALS(tag.tag_id)

    // Calculate current rate
    metal_rate := GET_CURRENT_METAL_RATE(product.metal_type, purity.id)

    RETURN {
        status: "SUCCESS",
        tag: tag,
        product: product,
        design: design,
        purity: purity,
        stones: stones,
        charges: charges,
        other_metals: other_metals,
        current_rate: metal_rate
    }
END FUNCTION
```

6.3.3 Item Cost Calculation

```
#ID: ESTIMATION.ADD.CALCULATE_ITEM_COST
#META: {"inputs":["item_details","rate","settings"],"outputs":["calculated_cost"],"critical":true}

FUNCTION CalculateEstimationItemCost(item, rate, settings) -> calculated_cost
```

```
PRECONDITION: Item has valid weight and purity
POSTCONDITION: Accurate cost calculated per business rules
SIDE_EFFECTS: None
FAILURE_MODES: Invalid rate, missing purity mapping

BEGIN
    // Determine weight for calculation
    IF item.calculation_based_on = "NET_WEIGHT" THEN
        calc_weight := item.net_wt
    ELSE
        calc_weight := item.gross_wt
    END IF

    // Base metal value
    metal_value := calc_weight * rate.rate_per_gram

    // Wastage calculation
    IF settings.wastage_rate_type = "PERCENTAGE" THEN
        wastage_value := metal_value * (item.wastage_percent / 100)
    ELSE // Weight-based
        wastage_weight := calc_weight * (item.wastage_percent / 100)
        wastage_value := wastage_weight * rate.rate_per_gram
    END IF

    // Making charge calculation
    IF item.mc_type = "PER_GRAM" THEN
        mc_value := calc_weight * item.mc_value
    ELSE IF item.mc_type = "FIXED" THEN
        mc_value := item.mc_value
    ELSE IF item.mc_type = "PERCENTAGE" THEN
        mc_value := metal_value * (item.mc_value / 100)
    END IF

    // Stone value
    stone_value := 0
    FOR EACH stone IN item.stones DO
        IF stone.cal_type = "PER_PIECE" THEN
            stone_value := stone_value + (stone.pieces * stone.rate)
        ELSE IF stone.cal_type = "PER_CARAT" THEN
            stone_value := stone_value + (stone.weight * stone.rate)
        ELSE // Fixed
            stone_value := stone_value + stone.price
        END IF
    END FOR

    // Other charges
    charges_value := SUM(item.charges.value)

    // Other metals value
    other_metals_value := 0
    FOR EACH om IN item.other_metals DO
        om_rate := GET_RATE_FOR_METAL(om.metal_id, om.purity_id)
        other_metals_value := other_metals_value + (om.weight * om_rate)
```

```
    END FOR

    // Subtotal before tax
    subtotal := metal_value + wastage_value + mc_value + stone_value + charges_value +
other_metals_value

    // Apply line discount if any
    IF item.discount > 0 THEN
        subtotal := subtotal - item.discount
    END IF

    // Tax calculation
    tax_details := GET_TAX_GROUP(item.product.tax_group_id)
    tax_amount := 0
    FOR EACH tax IN tax_details DO
        IF tax.calculation = "ON_TOTAL" THEN
            tax_amount := tax_amount + (subtotal * (tax.percentage / 100))
        END IF
    END FOR

    total := subtotal + tax_amount

    RETURN {
        metal_value: metal_value,
        wastage_value: wastage_value,
        mc_value: mc_value,
        stone_value: stone_value,
        charges_value: charges_value,
        other_metals_value: other_metals_value,
        subtotal: subtotal,
        tax_amount: tax_amount,
        total: ROUND(total, 2)
    }
END FUNCTION
```

6.3.4 Old Metal (Purchase) Processing

```
#ID: ESTIMATION.ADD.PROCESS_OLD_METAL
#META: {"inputs":["old_metal_details","settings"],"outputs":["purchase_record"],"critical":true}

FUNCTION ProcessOldMetalExchange(old_metal, settings) -> purchase_record

PRECONDITION: Old metal type and category valid
POSTCONDITION: Purchase value calculated correctly
SIDE_EFFECTS: None
FAILURE_MODES: Rate outside allowed limits, invalid purity

BEGIN
    // Validate rate within limits
    IF old_metal.metal_type = "GOLD" THEN
        IF old_metal.rate < settings.min_old_gold_rate THEN
            RAISE ERROR "Gold rate below minimum allowed"
```

```
        END IF
        IF old_metal.rate > settings.max_old_gold_rate THEN
            RAISE ERROR "Gold rate above maximum allowed"
        END IF
    ELSE IF old_metal.metal_type = "SILVER" THEN
        IF old_metal.rate < settings.min_old_silver_rate THEN
            RAISE ERROR "Silver rate below minimum allowed"
        END IF
        IF old_metal.rate > settings.max_old_silver_rate THEN
            RAISE ERROR "Silver rate above maximum allowed"
        END IF
    END IF

    // Calculate net weight (gross - stone weight - dust)
    net_weight := old_metal.gross_wt - old_metal.stone_wt - old_metal.dust_wt

    // Apply wastage deduction
    wastage_weight := net_weight * (old_metal.wastage_percent / 100)
    final_weight := net_weight - wastage_weight

    // Calculate purchase amount
    purchase_amount := final_weight * old_metal.rate_per_gram

    // Process stones in old metal (if reusable)
    stone_credit := 0
    FOR EACH stone IN old_metal.stones DO
        stone_credit := stone_credit + stone.value
    END FOR

    total_credit := purchase_amount + stone_credit

    RETURN {
        gross_wt: old_metal.gross_wt,
        stone_wt: old_metal.stone_wt,
        dust_wt: old_metal.dust_wt,
        net_wt: net_weight,
        wastage_wt: wastage_weight,
        final_wt: final_weight,
        rate_per_gram: old_metal.rate_per_gram,
        purchase_amount: purchase_amount,
        stone_credit: stone_credit,
        total_credit: ROUND(total_credit, 2),
        purpose: old_metal.purpose, // "MELTING" or "RETAG"
        item_type: old_metal.item_type // "ORNAMENT", "COIN", "BAR"
    }
END FUNCTION
```

6.3.5 Save Estimation

```
#ID: ESTIMATION.ADD.SAVE_ESTIMATION
#META: {"inputs":["estimation_data","items","old_metal","vouchers","chits"],"outputs":
["estimation_id"],"critical":true}
```

```
FUNCTION SaveEstimation(data, items, old_metal, vouchers, chits) -> estimation_id

PRECONDITION: All validations passed, branch day open
POSTCONDITION: Estimation and all child records persisted
SIDE_EFFECTS: Tag statuses updated (non-EDA), estimation number generated
FAILURE_MODES: Transaction failure, duplicate tag in estimation

BEGIN
    // Get branch day closing status
    branch_data := GET_BRANCH_DAY_STATUS(data.branch_id)
    IF branch_data IS NULL OR branch_data.is_day_closed = TRUE THEN
        RAISE ERROR "Cannot create estimation - day is closed"
    END IF

    // Validate all tags have prices
    FOR EACH item IN items WHERE item.type = "TAG" DO
        IF item.tax_price IS EMPTY OR item.tax_price = 0 THEN
            RAISE ERROR "All tagged items must have calculated price"
        END IF
    END FOR

    // Determine estimation datetime
    IF branch_data.entry_date = TODAY() THEN
        est_datetime := CURRENT_TIMESTAMP()
    ELSE
        est_datetime := branch_data.entry_date + CURRENT_TIME()
    END IF

    // Generate estimation number
    est_no := GENERATE_ESTIMATION_NUMBER(branch_data.entry_date, data.branch_id)
    fin_year := GET_ACTIVE_FINANCIAL_YEAR()

    TRANSACTION BEGIN

        // Insert master estimation record
        estimation := {
            estimation_datetime: est_datetime,
            fin_year_code: fin_year.code,
            esti_no: est_no,
            esti_for: data.esti_for, // 1=Customer, 2=Branch Transfer
            cus_id: data.customer_id,
            disc_per: data.discount_percent,
            bulk_was_disc_per: data.bulk_discount_percent,
            gift_voucher_amt: data.gift_voucher_amount,
            total_cost: data.total_cost,
            est_date: est_datetime,
            created_time: CURRENT_TIMESTAMP(),
            created_by: data.employee_id,
            id_branch: data.branch_id,
            is_eda: data.requires_eda,
            goldrate_22ct: data.gold_rate_22ct,
            silverrate_1gm: data.silver_rate_1gm,
```

```
    manual_rate: data.manual_rate,
    goldrate_18ct: data.gold_rate_18ct
}
estimation_id := INSERT("ret_estimation", estimation)

// Insert tagged items
FOR EACH tag_item IN items WHERE item_type = "TAG" DO
    item_record := BUILD_ESTIMATION_ITEM_RECORD(estimation_id, tag_item)
    item_id := INSERT("ret_estimation_items", item_record)

    // Insert stones for this item
    FOR EACH stone IN tag_item.stones DO
        stone_record := BUILD_STONE_RECORD(estimation_id, item_id, stone)
        INSERT("ret_estimation_item_stones", stone_record)
    END FOR

    // Insert other metals
    FOR EACH om IN tag_item.other_metals DO
        om_record := BUILD_OTHER_METAL_RECORD(item_id, om)
        INSERT("ret_est_other_metals", om_record)
    END FOR

    // Insert charges
    FOR EACH charge IN tag_item.charges DO
        charge_record := {
            est_item_id: item_id,
            id_charge: charge.charge_id,
            amount: charge.value
        }
        INSERT("ret_estimation_other_charges", charge_record)
    END FOR

    // Handle tag merge if applicable
    IF tag_item.has_merged_tags = TRUE THEN
        FOR EACH child_tag IN tag_item.merged_tags DO
            child_item_id := INSERT_CHILD_TAG_ITEM(estimation_id, child_tag)
            INSERT("ret_est_tag_merge", {
                est_item_id: item_id,
                ref_est_item_id: child_item_id
            })
            UPDATE("ret_estimation_items", item_id, {istag_merged: 1})
        END FOR
    END IF
END FOR

// Insert catalog items
FOR EACH catalog_item IN items WHERE item_type = "CATALOG" DO
    item_record := BUILD_CATALOG_ITEM_RECORD(estimation_id, catalog_item)
    item_id := INSERT("ret_estimation_items", item_record)
    SAVE_ITEM_STONES(item_id, catalog_item.stones)
    SAVE_ITEM_CHARGES(item_id, catalog_item.charges)
    SAVE_ITEM_MATERIALS(estimation_id, item_id, catalog_item.materials)
END FOR
```

```
// Insert custom items
FOR EACH custom_item IN items WHERE item_type = "CUSTOM" DO
    item_record := BUILD_CUSTOM_ITEM_RECORD(estimation_id, custom_item)
    item_id := INSERT("ret_estimation_items", item_record)
    SAVE_ITEM_STONES(item_id, custom_item.stones)
    SAVE_ITEM_CHARGES(item_id, custom_item.charges)
END FOR

// Insert order items
FOR EACH order_item IN items WHERE item_type = "ORDER" DO
    item_record := BUILD_ORDER_ITEM_RECORD(estimation_id, order_item)
    INSERT("ret_estimation_items", item_record)
    UPDATE("customerorder", order_item.order_no, {est_id: estimation_id})
END FOR

// Insert old metal purchase records
FOR EACH om IN old_metal DO
    om_record := BUILD_OLD_METAL_RECORD(estimation_id, om)
    om_id := INSERT("ret_estimation_old_metal_sale_details", om_record)

    // Insert stones in old metal
    FOR EACH stone IN om.stones DO
        INSERT("ret_esti_old_metal_stone_details", {
            est_old_metal_sale_id: om_id,
            stone_id: stone.stone_id,
            pieces: stone.pieces,
            wt: stone.weight,
            price: stone.price,
            rate_per_gram: stone.rate,
            is_apply_in_lwt: stone.apply_in_less_wt,
            stone_cal_type: stone.cal_type,
            uom_id: stone.uom_id
        })
    END FOR
END FOR

// Insert gift vouchers
FOR EACH voucher IN vouchers DO
    INSERT("ret_est_gift_voucher_details", {
        est_id: estimation_id,
        voucher_no: voucher.number,
        gift_voucher_details: voucher.details,
        gift_voucher_amt: voucher.amount
    })
END FOR

// Insert chit utilization
FOR EACH chit IN chits DO
    INSERT("ret_est_chit_utilization", {
        est_id: estimation_id,
        scheme_account_id: chit.account_id,
        utl_amount: chit.utilized_amount,
```

```
                closing_weight: chit.closing_weight,
                wastage_per: chit.wastage_percent,
                savings_in_wastage: chit.wastage_savings,
                mc_value: chit.mc_value,
                savings_in_making_charge: chit.mc_savings,
                rate_per_gram: chit.rate_per_gram
            })
        END FOR

    TRANSACTION COMMIT

    // Log activity
    LOG_DETAIL({
        module: "Estimation",
        operation: "Create",
        record: estimation_id,
        remark: "Estimation created successfully"
    })

    RETURN {
        status: TRUE,
        estimation_id: estimation_id,
        estimation_no: est_no
    }

EXCEPTION
    TRANSACTION ROLLBACK
    RETURN {
        status: FALSE,
        message: exception.message
    }
END FUNCTION
```

6.4 Business Rules and Validations

Rule ID	Rule Description	Enforcement Point
ADD-001	Branch day must be open	Save Estimation
ADD-002	All tagged items must have tax_price > 0	Save Estimation
ADD-003	Old gold rate must be within min/max limits	Old Metal Processing
ADD-004	Old silver rate must be within min/max limits	Old Metal Processing
ADD-005	Tag must be in available status (1,3,4,6)	Tag Search
ADD-006	Tag must belong to selected branch	Tag Search

Rule ID	Rule Description	Enforcement Point
ADD-007	Employee discount cannot exceed settings limit	Discount Application
ADD-008	Customer required for customer-type estimation	Form Validation
ADD-009	At least one item required	Save Estimation
ADD-010	Manual rate change requires employee permission	Rate Change

6.5 Exception & Edge Scenarios

```
#ID: ESTIMATION.ADD.HANDLE_DAY_CLOSED
#META: {"inputs":["branch_id"],"outputs":["error_response"],"critical":false}

FUNCTION HandleDayClosedScenario(branch_id) -> error_response
BEGIN
    // Check if day can be reopened
    last_closing := GET_LAST_DAY_CLOSING(branch_id)

    IF last_closing.entry_date < TODAY() - 1 THEN
        RETURN {
            error_type: "DAY_CLOSED_PAST",
            message: "Cannot create estimation for past dates",
            recovery: "Contact admin to reopen day"
        }
    ELSE
        RETURN {
            error_type: "DAY_CLOSED_TODAY",
            message: "Day is closed for selected branch",
            recovery: "Open today's day or select different branch"
        }
    END IF
END FUNCTION
```

```
#ID: ESTIMATION.ADD.HANDLE_TAG_UNAVAILABLE
#META: {"inputs":["tag_id","tag_status"],"outputs":["error_response"],"critical":false}

FUNCTION HandleTagUnavailableScenario(tag_id, tag_status) -> error_response
BEGIN
    status_messages := {
        9: "Tag is already in another estimation",
        10: "Tag has been sold",
        11: "Tag is in branch transfer",
        12: "Tag is locked for approval"
    }

    RETURN {
        error_type: "TAG_UNAVAILABLE",
```

```
        message: status_messages[tag_status],
        tag_id: tag_id,
        recovery: "Select a different tag or contact manager"
    }
END FUNCTION
```

---

7. Sub-Module: Estimation List

Route: `admin_ret_estimation/estimation/list`

7.1 Purpose

Provides a searchable, filterable list view of all estimations. Enables viewing, editing, printing, and tracking of estimation-to-bill conversions.

7.2 Functional Actors

Actor	Actions
Sales Staff	View own estimations, filter by date/branch, print copies
Branch Manager	View all branch estimations, access edit functions
System	Load data, format bill numbers, calculate display values

7.3 Functional Workflow

7.3.1 Load Estimation List

```
#ID: ESTIMATION.LIST.LOAD_LIST
#META: {"inputs":["branch_id","from_date","to_date"],"outputs":["estimation_list"],"critical":false}

FUNCTION LoadEstimationList(branch_id, from_date, to_date) -> estimation_list

PRECONDITION: User has estimation/list permission
POSTCONDITION: Filtered estimation list returned
SIDE_EFFECTS: None
FAILURE_MODES: Database query timeout

BEGIN
    // Determine date range
    IF from_date IS EMPTY OR to_date IS EMPTY THEN
        IF branch_id > 0 THEN
            branch_data := GET_BRANCH_DAY_STATUS(branch_id)
            filter_date := branch_data.entry_date
```

```
    ELSE
        filter_date := TODAY()
    END IF
    date_filter := "date(estimation_datetime) = '" + filter_date + "'"
ELSE
    date_filter := "date(estimation_datetime) BETWEEN '" + from_date + "' AND '" + to_date + "'"
END IF

// Build query
estimations := QUERY
    SELECT
        est.estimation_id,
        est.esti_no,
        customer.mobile,
        FORMAT_DATETIME(est.estimation_datetime) as estimation_datetime,
        est.total_cost,
        CASE est.esti_for WHEN 1 THEN 'Customer' ELSE 'Branch Transfer' END as esti_for,
        COALESCE(bill.bill_id, old_bill.bill_id) as bill_id,
        COALESCE(bill.bill_no, old_bill.bill_no) as bill_no,
        chit_util.chit_ut_id,
        sr_util.sr_ut_id,
        product_concat.product_name,
        customer.firstname,
        employee.firstname as emp_name,
        employee.emp_code,
        review.rating,
        review.review,
        review.suggestion,
        CASE est.added_through WHEN 1 THEN 'Admin' WHEN 2 THEN 'App' ELSE '' END as added_through,
        scheme.is_topup_scheme
    FROM ret_estimation est
    LEFT JOIN customer ON customer.id_customer = est.cus_id
    LEFT JOIN employee ON employee.id_employee = est.created_by
    LEFT JOIN ret_customer_review review ON review.esti_id = est.estimation_id
    LEFT JOIN ret_est_chit_utilization chit_util ON chit_util.est_id = est.estimation_id
    LEFT JOIN scheme_account sa ON sa.id_scheme_account = chit_util.scheme_account_id
    LEFT JOIN scheme ON scheme.id_scheme = sa.id_scheme
    LEFT JOIN ret_est_sales_return_utilization sr_util ON sr_util.est_id = est.estimation_id
    LEFT JOIN (...bill subquery...) bill ON bill.estimation_id = est.estimation_id
    LEFT JOIN (...product subquery...) product_concat ON product_concat.esti_id = est.estimation_id
    LEFT JOIN (...old_bill subquery...) old_bill ON old_bill.est_id = est.estimation_id
    WHERE date_filter
    AND (branch_id = 0 OR est.id_branch = branch_id)
    GROUP BY est.estimation_id
    ORDER BY est.estimation_id DESC

// Post-process results
result_list := []
FOR EACH est IN estimations DO
    // Determine item type
    old_metal_count := GET_OLD_METAL_COUNT(est.estimation_id)
    IF old_metal_count > 0 THEN
        est.item_type := "Sales and Purchase"
```

```
        ELSE
            est.item_type := "Sales"
        END IF

        // Format bill number if exists
        IF est.bill_id IS NOT EMPTY THEN
            est.bill_no := FORMAT_BILL_NUMBER(est.bill_id)
        END IF

        APPEND result_list, est
    END FOR

    RETURN result_list
END FUNCTION
```

7.3.2 View Estimation Details

```
#ID: ESTIMATION.LIST.VIEW_DETAILS
#META: {"inputs":["estimation_id"],"outputs":["estimation_full_record"],"critical":false}

FUNCTION ViewEstimationDetails(estimation_id) -> estimation_full_record

PRECONDITION: Estimation ID valid
POSTCONDITION: Complete estimation data returned
SIDE_EFFECTS: None
FAILURE_MODES: Estimation not found

BEGIN
    // Get master record
    estimation := GET_ENTRY_RECORD(estimation_id)
    IF estimation IS NULL THEN
        RAISE ERROR "Estimation not found"
    END IF

    // Get all related data
    other_details := GET_OTHER_ESTIMATION_ITEMS(estimation_id)

    RETURN {
        master: estimation,
        items: other_details.item_details,
        old_metal: other_details.old_matel_details,
        stones: other_details.stone_details,
        materials: other_details.other_material_details,
        vouchers: other_details.voucher_details,
        chits: other_details.chit_details,
        sales_returns: other_details.sales_return_details,
        advances: other_details.advance_details
    }
END FUNCTION
```

7.3.3 Generate Invoice PDF

```
#ID: ESTIMATION.LIST.GENERATE_INVOICE
#META: {"inputs":["estimation_id"],"outputs":["pdf_file"],"critical":false}

FUNCTION GenerateEstimationInvoice(estimation_id) -> pdf_file

PRECONDITION: Estimation exists
POSTCONDITION: PDF generated and returned
SIDE_EFFECTS: None
FAILURE_MODES: PDF generation failure

BEGIN
    // Load estimation data
    est_data := VIEW_ESTIMATION_DETAILS(estimation_id)

    // Load print settings
    print_settings := GET_PRINT_SETTINGS()

    // Build HTML content
    html_content := RENDER_TEMPLATE("estimation/print/invoice", {
        estimation: est_data,
        settings: print_settings,
        company: GET_COMPANY_DETAILS()
    })

    // Generate PDF using DOMPDF
    pdf := NEW DOMPDF()
    pdf.loadHtml(html_content)
    pdf.setPaper(print_settings.paper_size, print_settings.orientation)
    pdf.render()

    RETURN pdf.output()
END FUNCTION
```

7.4 Business Rules and Validations

Rule ID	Rule Description	Enforcement Point
LIST-001	Default to current day if no date filter	List Load
LIST-002	Branch filter applies when branch_id > 0	List Load
LIST-003	Bill number formatting per bill*no*format settings	Result Processing
LIST-004	Estimations with bill_id are non-editable	UI Display

7.5 Exception & Edge Scenarios

```
#ID: ESTIMATION.LIST.HANDLE_NO_DATA
#META: {"inputs":["filters"],"outputs":["empty_state"],"critical":false}

FUNCTION HandleNoDataScenario(filters) -> empty_state
BEGIN
    RETURN {
        error_type: "NO_DATA",
        message: "No estimations found for the selected criteria",
        filters_applied: filters,
        suggestions: [
            "Try widening the date range",
            "Check if correct branch is selected",
            "Verify estimations exist for this period"
        ]
    }
END FUNCTION
```

---

8. Sub-Module: Estimate Discount Approval (EDA)

Route: `admin_ret_eda/eda/list`

8.1 Purpose

Provides a manager approval workflow for estimations that contain discounts exceeding employee limits. Enables approve/reject decisions with final amount adjustments.

8.2 Functional Actors

Actor	Actions
Branch Manager	View pending EDAs, approve with final amount, reject with reason
System	Update estimation status, update tag statuses, log approvals

8.3 Functional Workflow

8.3.1 Load EDA List

```
#ID: ESTIMATION.EDA.LOAD_LIST
#META: {"inputs":["branch_id"],"outputs":["eda_list"],"critical":false}

FUNCTION LoadEDAList(branch_id) -> eda_list

PRECONDITION: User has eda/list permission
POSTCONDITION: Pending EDA estimations returned
```

```
SIDE_EFFECTS: None
FAILURE_MODES: None

BEGIN
    // Query pending EDA estimations
    eda_list := QUERY
        SELECT
            est.estimation_id,
            est.esti_no,
            FORMAT_DATETIME(est.estimation_datetime) as estimation_datetime,
            customer.firstname as customer_name,
            customer.mobile,
            est.total_cost as estimate_total_amt,
            est.estimate_final_amt,
            (est.total_cost - COALESCE(est.estimate_final_amt, est.total_cost)) as
estimate_discount_amt,
            est.disc_per,
            est.is_eda_approved,
            employee.firstname as emp_name,
            employee.emp_code,
            branch.branch_name,
            product_list.products
        FROM ret_estimation est
        LEFT JOIN customer ON customer.id_customer = est.cus_id
        LEFT JOIN employee ON employee.id_employee = est.created_by
        LEFT JOIN branch ON branch.id_branch = est.id_branch
        LEFT JOIN (...product aggregation...) product_list ON product_list.esti_id = est.estimation_id
        WHERE est.is_eda = 1
        AND est.is_eda_approved = 0  -- Pending approval
        AND (branch_id = 0 OR est.id_branch = branch_id)
        ORDER BY est.estimation_id DESC

    RETURN eda_list
END FUNCTION
```

8.3.2 Approve EDA

```
#ID: ESTIMATION.EDA.APPROVE
#META: {"inputs":["estimation_id","final_amount"],"outputs":["approval_result"],"critical":true}

FUNCTION ApproveEDA(estimation_id, final_amount) -> approval_result

PRECONDITION: Estimation exists with is_eda=1 and is_eda_approved=0
POSTCONDITION: Estimation approved, tags marked as sold (status=10)
SIDE_EFFECTS: Tag statuses updated, non-tag inventory adjusted, log entry created
FAILURE_MODES: Tag not found, transaction failure

BEGIN
    // Get estimation details
    est_details := GET_ESTIMATION_DETAILS(estimation_id)
    IF est_details IS NULL OR LENGTH(est_details) = 0 THEN
        RETURN {status: FALSE, message: "Tag Details Not Found"}
```

```
END IF

TRANSACTION BEGIN

    // Update estimation approval status
    UPDATE("ret_estimation", estimation_id, {
        estimate_final_amt: final_amount,
        is_eda_approved: 1
    })

    // Process each item
    FOR EACH item IN est_details DO

        IF item.tag_id IS NOT EMPTY THEN
            // Tagged item - update tag status to SOLD (10)
            UPDATE("ret_taging", item.tag_id, {tag_status: 10})

            // Log tag status change
            INSERT("ret_taging_status_log", {
                tag_id: item.tag_id,
                date: CURRENT_TIMESTAMP(),
                status: 10,
                from_branch: item.id_branch,
                to_branch: NULL,
                created_on: CURRENT_TIMESTAMP(),
                created_by: CURRENT_USER_ID()
            })

        ELSE IF item.is_non_tag = 1 THEN
            // Non-tagged item - update inventory
            exist_data := {
                id_product: item.product_id,
                id_design: item.design_id,
                id_branch: item.id_branch
            }

            exist_check := CHECK_NON_TAG_ITEM_EXISTS(exist_data)

            IF exist_check.status = TRUE THEN
                // Reduce inventory
                nt_update := {
                    id_nontag_item: exist_check.id_nontag_item,
                    no_of_piece: item.piece,
                    gross_wt: item.gross_wt,
                    net_wt: item.net_wt,
                    less_wt: item.less_wt,
                    updated_by: CURRENT_USER_ID(),
                    updated_on: CURRENT_TIMESTAMP()
                }
                UPDATE_NON_TAG_DATA(nt_update, operation: "SUBTRACT")

                // Log non-tag movement
                INSERT("ret_nontag_item_log", {
```

```
                        from_branch: item.id_branch,
                        to_branch: NULL,
                        no_of_piece: item.piece,
                        less_wt: item.less_wt,
                        net_wt: item.net_wt,
                        gross_wt: item.gross_wt,
                        product: item.product_id,
                        design: item.design_id,
                        date: item.est_date,
                        created_on: CURRENT_TIMESTAMP(),
                        created_by: CURRENT_USER_ID(),
                        status: 10,
                        bill_id: estimation_id
                    })
                END IF
            END IF
        END FOR

        // Check transaction success
        IF TRANSACTION_STATUS() = TRUE THEN
            TRANSACTION COMMIT

            // Log approval activity
            LOG_DETAIL({
                module: "EDA Estimation",
                operation: "Approval",
                record: estimation_id,
                remark: "Record Approved successfully"
            })

            RETURN {status: TRUE}
        ELSE
            TRANSACTION ROLLBACK
            RETURN {status: FALSE, message: "Unable to Proceed Your Request"}
        END IF

    EXCEPTION
        TRANSACTION ROLLBACK
        RETURN {status: FALSE, message: exception.message}
    END TRANSACTION

END FUNCTION
```

8.3.3 Reject EDA

```
#ID: ESTIMATION.EDA.REJECT
#META: {"inputs":["estimation_id"],"outputs":["rejection_result"],"critical":true}

FUNCTION RejectEDA(estimation_id) -> rejection_result

PRECONDITION: Estimation exists with is_eda=1 and is_eda_approved=0
POSTCONDITION: Estimation marked as rejected (is_eda_approved=2)
```

```
SIDE_EFFECTS: Log entry created
FAILURE_MODES: Update failure

BEGIN
    // Update estimation status to rejected
    update_result := UPDATE("ret_estimation", estimation_id, {
        is_eda_approved: 2
    })

    IF update_result = TRUE THEN
        // Log rejection
        LOG_DETAIL({
            module: "EDA Estimation",
            operation: "Rejection",
            record: estimation_id,
            remark: "Record Rejected"
        })

        RETURN {status: TRUE}
    ELSE
        RETURN {status: FALSE, message: "Failed to update estimation"}
    END IF
END FUNCTION
```

8.4 Business Rules and Validations

Rule ID	Rule Description	Enforcement Point
EDA-001	Only estimations with is_eda=1 appear in EDA list	List Query
EDA-002	Only pending (is*eda*approved=0) shown by default	List Query
EDA-003	Approval requires valid final_amount	Approve Function
EDA-004	Approval updates all associated tag statuses to 10	Approve Processing
EDA-005	Non-tag items reduce inventory on approval	Approve Processing
EDA-006	Rejection sets is*eda*approved=2	Reject Function
EDA-007	Rejected estimations can be edited and resubmitted	UI Logic

8.5 Exception & Edge Scenarios

```
#ID: ESTIMATION.EDA.HANDLE_TAG_NOT_FOUND
#META: {"inputs":["estimation_id"],"outputs":["error_response"],"critical":false}

FUNCTION HandleTagNotFoundOnApproval(estimation_id) -> error_response
```

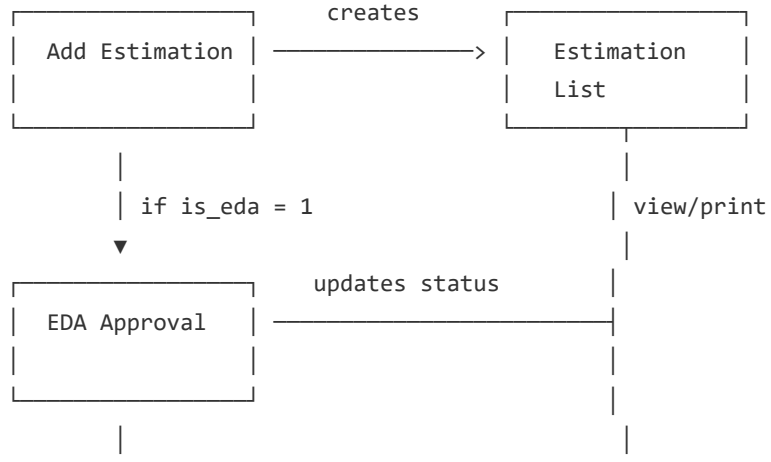
```
BEGIN
    RETURN {
        error_type: "TAG_NOT_FOUND",
        message: "One or more tags in this estimation could not be found",
        estimation_id: estimation_id,
        recovery: [
            "Verify estimation items are still valid",
            "Check if tags were deleted or transferred",
            "Contact system administrator"
        ]
    }
END FUNCTION
```

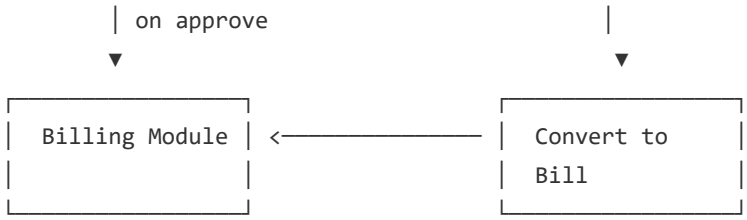
```
#ID: ESTIMATION.EDA.HANDLE_ALREADY_PROCESSED
#META: {"inputs":["estimation_id","current_status"],"outputs":["error_response"],"critical":false}

FUNCTION HandleAlreadyProcessed(estimation_id, current_status) -> error_response
BEGIN
    status_names := {
        1: "already approved",
        2: "already rejected"
    }

    RETURN {
        error_type: "ALREADY_PROCESSED",
        message: "This estimation has been " + status_names[current_status],
        estimation_id: estimation_id,
        current_status: current_status,
        recovery: "Refresh the list to see updated status"
    }
END FUNCTION
```

---

9. Inter Sub-Module Dependency Flow

9.1 Data Flow Between Sub-Modules





9.2 Status Transitions

From State	Action	To State	Sub-Module
(new)	Create Estimation	Active	Add Estimation
Active (non-EDA)	Create	Tags status=9	Add Estimation
Active (EDA)	Create	is*eda*approved=0	Add Estimation
Pending EDA	Approve	is*eda*approved=1, tags=10	EDA
Pending EDA	Reject	is*eda*approved=2	EDA
Rejected	Re-edit	Pending EDA	Add Estimation
Approved	Convert to Bill	estbillid populated	Billing

9.3 Rejection Handling

When an EDA is rejected: 1. `is_eda_approved` set to 2 2. Estimation remains editable via Edit function 3. User can modify discounts and resubmit 4. On resubmit, status returns to pending (is*eda*approved=0)

9.4 Skip/Delay Handling

Scenario	System Behavior
EDA pending but billing attempted	Blocked - estimation must be approved first
Day closed after estimation created	Estimation remains, cannot create new
Customer not found	Create inline from estimation form

---

10. Common Functional Scenarios

10.1 Happy Path - Standard Sale

```
#ID: ESTIMATION.SCENARIO.HAPPY_PATH_SALE
#META: {"inputs":["customer","items"],"outputs":["bill"],"critical":false}

SCENARIO StandardSaleWithoutDiscount
BEGIN
    1. Sales staff opens Add Estimation
    2. Selects existing customer or creates new
    3. Scans tag code for jewelry item
    4. System calculates price based on:
       - Current metal rate
       - Wastage percentage from tag
       - Making charges from tag
       - Stone values
       - Tax
    5. Adds item to estimation
    6. Repeats for additional items
    7. Verifies totals
    8. Saves estimation (is_eda = 0)
    9. Tags updated to status = 9 (estimated)
    10. Print invoice for customer
    11. Convert to bill when customer pays
END SCENARIO
```

10.2 Discount Requiring Approval

```
#ID: ESTIMATION.SCENARIO.DISCOUNT_APPROVAL
#META: {"inputs":["customer","items","discount"],"outputs":["approved_estimation"],"critical":false}

SCENARIO DiscountExceedingLimit
BEGIN
    1. Sales staff creates estimation with items
    2. Applies discount exceeding personal limit
    3. System sets is_eda = 1 on save
    4. Estimation saved with is_eda_approved = 0
    5. Tags NOT updated (remain available)
    6. Estimation appears in manager's EDA queue
    7. Manager reviews:
       - Total amount
       - Discount requested
       - Customer details
    8. Manager adjusts final amount (optional)
    9. Manager approves
    10. System updates:
        - is_eda_approved = 1
        - Tags status = 10 (sold)
        - Non-tag inventory reduced
    11. Estimation ready for billing
END SCENARIO
```

10.3 Old Metal Exchange

```
#ID: ESTIMATION.SCENARIO.OLD_METAL_EXCHANGE
#META: {"inputs":["customer","sale_items","old_metal"],"outputs":["net_estimation"],"critical":false}

SCENARIO SaleWithOldMetalExchange
BEGIN
    1. Customer brings old gold ornament
    2. Sales staff creates estimation with new items
    3. Opens Old Metal section
    4. Enters:
       - Metal type (Gold/Silver)
       - Item type (Ornament/Coin/Bar)
       - Gross weight
       - Stone weight (if any)
       - Dust/impurity weight
       - Purity
       - Purpose (Melting/Retag)
    5. System calculates:
       - Net weight = Gross - Stone - Dust
       - Wastage weight
       - Final weight
       - Purchase amount at current old metal rate
    6. Sale amount reduced by purchase credit
    7. Net amount = Sale - Purchase credit
    8. Saves estimation
    9. Both sale and purchase tracked
END SCENARIO
```

10.4 Rejection and Rework

```
#ID: ESTIMATION.SCENARIO.REJECTION_REWORK
#META: {"inputs":["rejected_estimation"],"outputs":["reapproved_estimation"],"critical":false}

SCENARIO RejectionAndResubmission
BEGIN
    1. Manager rejects EDA (excessive discount)
    2. is_eda_approved set to 2
    3. Sales staff opens rejected estimation
    4. Modifies discount within acceptable range
    5. Saves changes
    6. is_eda_approved reset to 0
    7. Estimation returns to EDA queue
    8. Manager approves revised request
    9. Normal approval flow continues
END SCENARIO
```

---

11. Operational Notes for Developers & Release Engineers

11.1 Troubleshooting Guide

Symptom	Likely Cause	Resolution
"Day is closed" error	Branch day not opened	Open day via Day Closing module
Tag not found in search	Wrong branch or tag status	Verify tag branch and status in ret_taging
Rate showing as 0	No rate entry for date/metal	Check ret*metal*rate for current date
EDA not appearing in queue	is_eda not set to 1	Verify discount exceeds limit in employee_settings
PDF generation fails	DOMPDF library issue	Check dompdf autoload path

11.2 Misconfiguration Patterns

Setting	Issue When Misconfigured
`min_old_gold_rate` = 0	Allows purchase at any rate
`bulk_wastage_discount` disabled	Bulk discount field hidden but may have data
Employee `disc_limit` = 0	All discounts require EDA
Branch not in `ret_day_closing`	All estimations fail

11.3 Release Checkpoints

- [ ] Verify `ret_settings` has all required keys
- [ ] Verify `employee_settings` has discount limits for all users
- [ ] Verify `ret_day_closing` has entries for all branches
- [ ] Verify `ret_metal_rate` has current date rates
- [ ] Verify `bill_no_format` configured for estimation types
- [ ] Test tag scan with various tag statuses
- [ ] Test old metal within and outside rate limits
- [ ] Test EDA approval/rejection cycle

11.4 Recommended Test Scenarios

1. Create estimation with single tagged item

2. Create estimation with catalog + custom items
3. Create estimation with old metal exchange
4. Create estimation with discount exceeding limit (triggers EDA)
5. Approve EDA and verify tag statuses
6. Reject EDA and verify resubmission flow
7. Create estimation with merged tags
8. Create estimation with scheme/chit utilization
9. Generate and verify PDF invoice
10. Convert estimation to bill

---

12. Pseudocode-to-Code Mapping Appendix

Pseudocode Function ID	File	Method/Function
ESTIMATION.CORE.MAIN_PROCESS	admin*ret*estimation.php	estimation('save')
ESTIMATION.ADD.INITIALIZE_FORM	admin*ret*estimation.php	estimation('add')
ESTIMATION.ADD.SEARCH_TAG	ret*estimation*model.php	getTaggingBySearch()
ESTIMATION.ADD.CALCULATE*ITEM*COST	ret_estimation.js	calculateTagCost()
ESTIMATION.ADD.PROCESS*OLD*METAL	ret_estimation.js	calculateOldMetalValue()
ESTIMATION.ADD.SAVE_ESTIMATION	admin*ret*estimation.php	estimation('save')
ESTIMATION.LIST.LOAD_LIST	ret*estimation*model.php	ajax_getEstimationList()
ESTIMATION.LIST.VIEW_DETAILS	ret*estimation*model.php	get*entry*records(), getOtherEstimateItemsDetails()
ESTIMATION.LIST.GENERATE_INVOICE	admin*ret*estimation.php	generate_invoice()
ESTIMATION.EDA.LOAD_LIST	ret*eda*model.php	ajax_getEdaList()
ESTIMATION.EDA.APPROVE	admin*ret*eda.php	eda('update') type=1
ESTIMATION.EDA.REJECT	admin*ret*eda.php	eda('update') type=2

---

13. Acceptance Criteria & Sign-off Checklist

13.1 Functional Validation

- [ ] Add Estimation form loads with all settings
- [ ] Tag search returns valid tags only
- [ ] Cost calculations match manual verification
- [ ] Old metal rates enforce min/max limits
- [ ] Discounts beyond limit trigger EDA
- [ ] EDA approval updates all tag statuses
- [ ] EDA rejection allows re-editing
- [ ] PDF generation produces valid output
- [ ] Estimation list filters correctly

13.2 Integration Validation

- [ ] Customer creation works inline
- [ ] Scheme/chit utilization deducts correctly
- [ ] Order linkage updates order status
- [ ] Billing conversion transfers all data

13.3 Sign-off Roles

Role	Responsibility
Developer	Code implementation matches pseudocode
QA	All test scenarios pass
Business Analyst	Business rules correctly implemented
Release Engineer	Deployment checklist complete

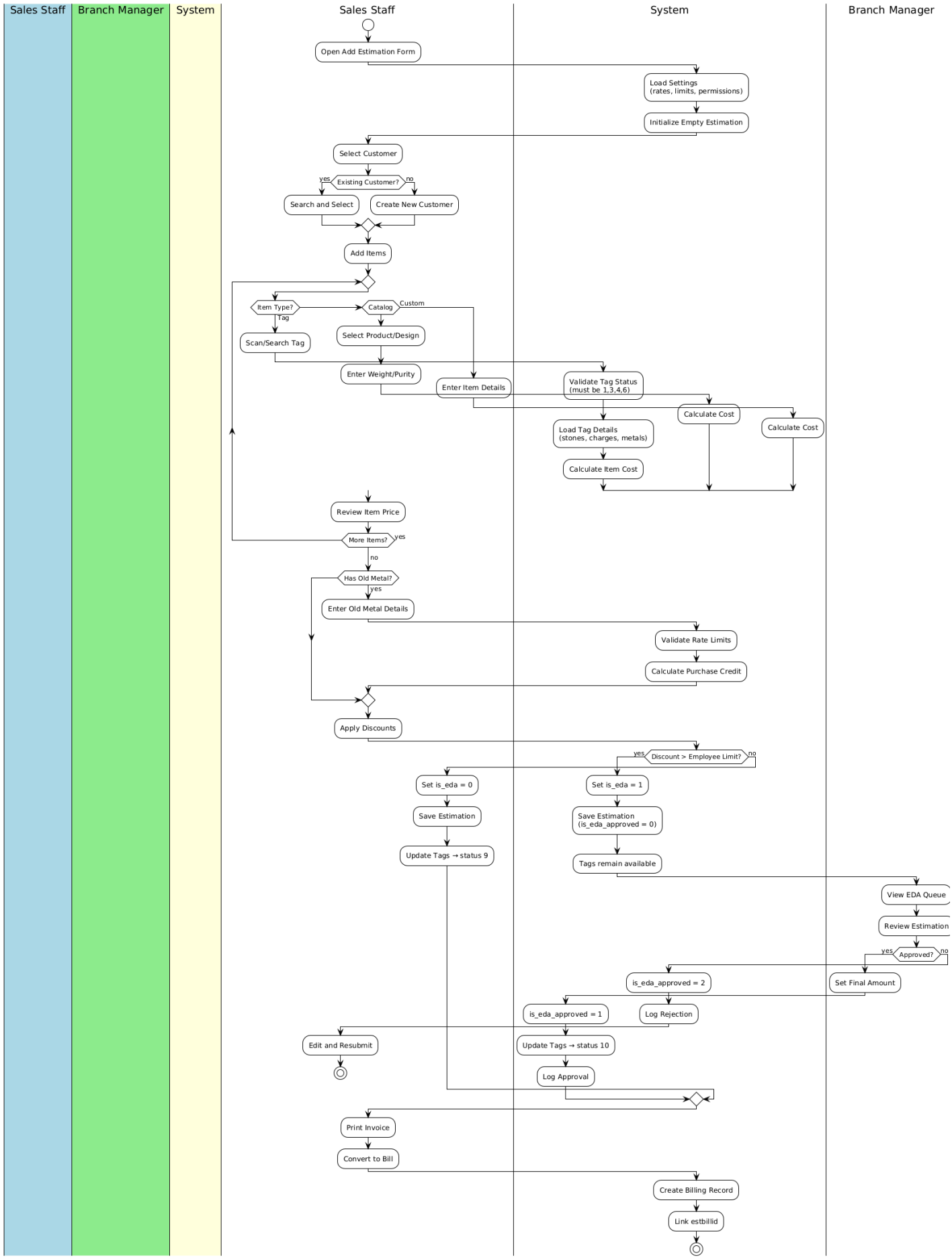
---

14. Change Log

Date	Author	Version	Changes
2026-01-20	Kanagasundar	1.0	Initial document creation

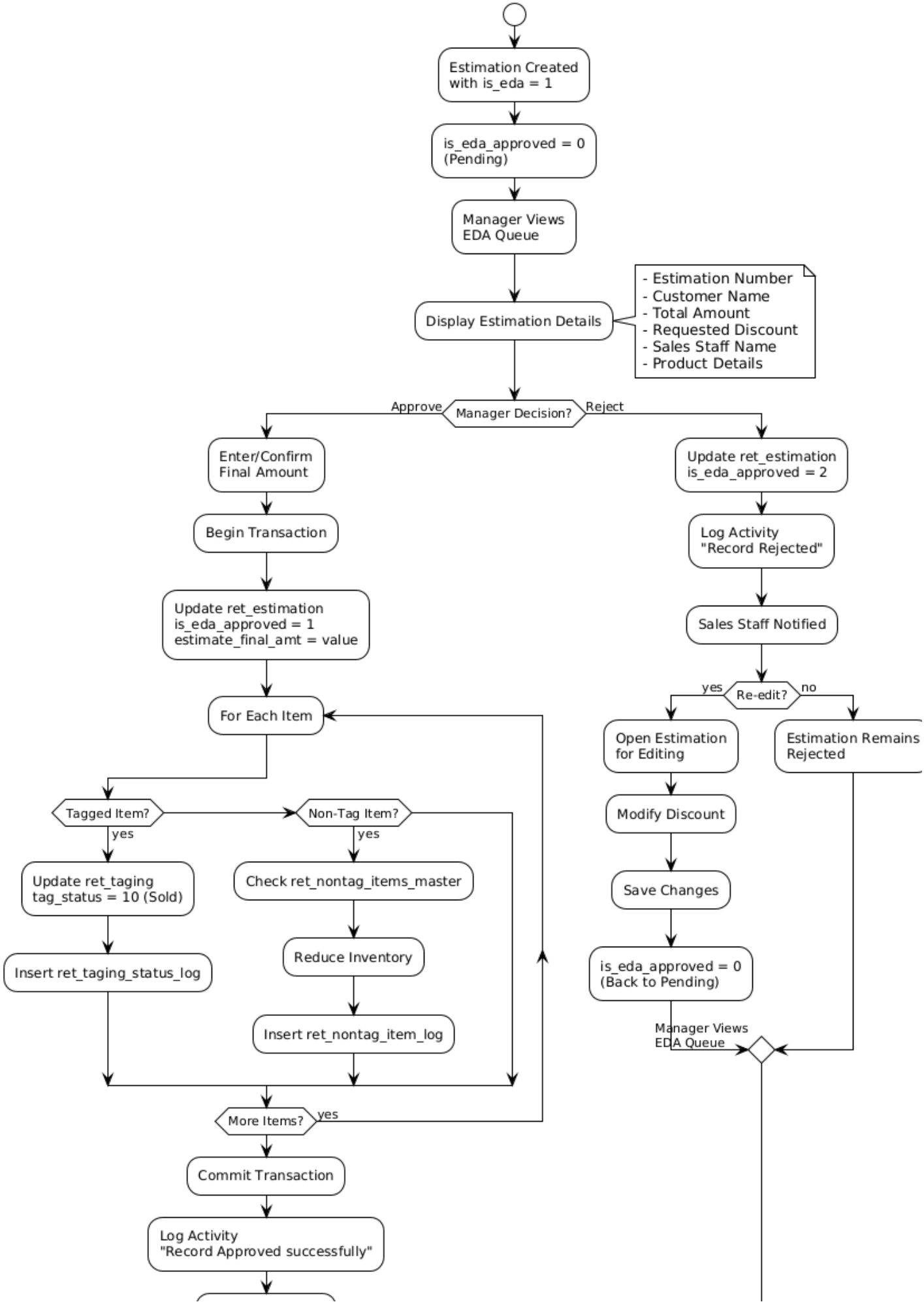
Document generated for Antigravity Knowledge Ingestion System Module: Estimation | Version: 1.0

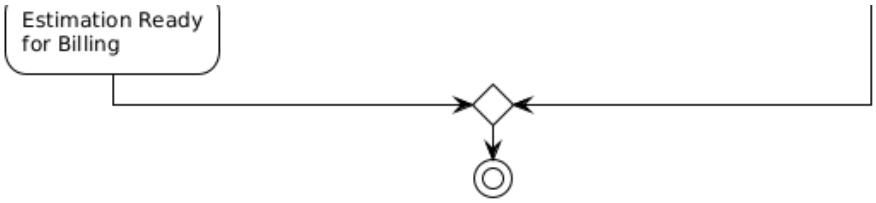
Appendix: Workflow Diagrams



Estimation Workflow

EDA Approval Decision Flow





Eda Approval Flow